

# L20 — Circular Linked List and Circular Doubly Linked List

Unit: 2 | Chapter: Linked Lists | BT Level: BT3 | CO: CO2

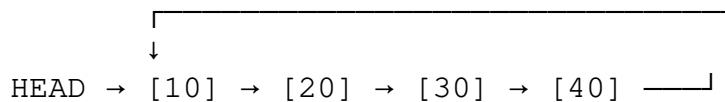
---

## Learning Objectives

- Explain the structure of circular singly and doubly linked lists
  - Implement insertion, deletion, and traversal for circular linked lists
  - Identify use cases where circular lists are preferred
  - Differentiate circular lists from their linear counterparts
- 

## 1. Circular Singly Linked List (CSLL)

In a **circular singly linked list**, the last node's `next` pointer points back to the **HEAD** (not **NULL**).



There is **no NULL** in the chain — the list wraps around.

---

## 2. CSLL Operations

### 2.1 Traversal — must detect when we're back at HEAD

```
def traverse_csll(head):  
    if head is None:  
        return  
    current = head  
    while True:  
        print(current.data, end=" → ")  
        current = current.next  
        if current == head:    # Full circle completed  
            break  
    print("(back to head)")
```

### 2.2 Insert at Beginning — O(n) (need to update last node)

```
def insert_beginning(self, data):  
    new_node = Node(data)  
    if self.head is None:  
        new_node.next = new_node    # Points to itself  
        self.head = new_node
```

```

        return

    # Find last node (the one pointing to head)
    last = self.head
    while last.next != self.head:
        last = last.next

    new_node.next = self.head
    last.next = new_node      # Last node now points to new head
    self.head = new_node

```

## 2.3 Insert at End — $O(n)$

```

def insert_end(self, data):
    new_node = Node(data)
    if self.head is None:
        new_node.next = new_node
        self.head = new_node
        return

    last = self.head
    while last.next != self.head:
        last = last.next

    last.next = new_node
    new_node.next = self.head    # New node circles back to head

```

## 2.4 CSLL with Tail Pointer — $O(1)$ insert at end

Using a **tail pointer** (instead of head) avoids  $O(n)$  traversal:

```

class CSLL_WithTail:
    def __init__(self):
        self.tail = None    # Points to last node

    def insert_end(self, data):
        new_node = Node(data)
        if self.tail is None:
            new_node.next = new_node
            self.tail = new_node
        else:
            new_node.next = self.tail.next    # New last → old first (he
            self.tail.next = new_node
            self.tail = new_node

    @property
    def head(self):
        return self.tail.next if self.tail else None

```

With tail pointer: insert at end →  $O(1)$ , insert at beginning →  $O(1)$ .

## 2.5 Delete a Node — $O(n)$

```

def delete_node(self, key):
    if self.head is None:
        return

    last = self.head
    while last.next != self.head:
        last = last.next

    # Deleting head
    if self.head.data == key:
        if self.head == last:    # Only one node
            self.head = None
        else:
            last.next = self.head.next
            self.head = self.head.next
        return

    # Search and delete non-head node
    curr = self.head
    while curr.next != self.head:
        if curr.next.data == key:
            curr.next = curr.next.next
            return
        curr = curr.next

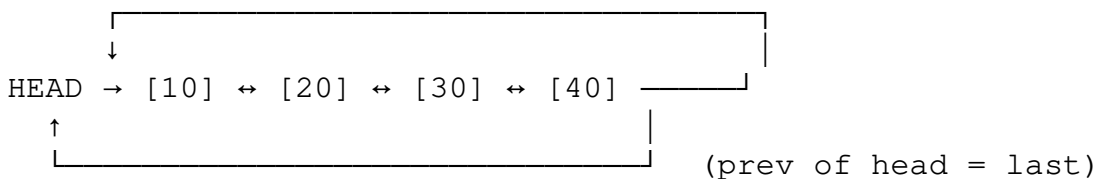
```

---

### 3. Circular Doubly Linked List (CDLL)

In a **CDLL**, each node has both `prev` and `next` pointers, and:

- `last.next = first`
- `first.prev = last`



#### 3.1 Node Structure

```

class CDLLNode:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

```

#### 3.2 CDLL with Sentinel (Header) Node

The cleanest implementation uses a **sentinel/dummy node**:

```

class CDLL:
    def __init__(self):

```

```

        self.sentinel = CDLLNode(None)    # Dummy node
        self.sentinel.next = self.sentinel
        self.sentinel.prev = self.sentinel

    def insert_after(self, node, data):
        """Insert new node after 'node' - O(1)"""
        new_node = CDLLNode(data)
        new_node.next = node.next
        new_node.prev = node
        node.next.prev = new_node
        node.next = new_node

    def insert_front(self, data):
        self.insert_after(self.sentinel, data)

    def insert_back(self, data):
        self.insert_after(self.sentinel.prev, data)

    def delete_node(self, node):
        """Remove a node - O(1) since we have prev pointer"""
        node.prev.next = node.next
        node.next.prev = node.prev

    def traverse_forward(self):
        curr = self.sentinel.next
        while curr != self.sentinel:
            print(curr.data, end=" ⇔ ")
            curr = curr.next
        print("(sentinel)")

    def traverse_backward(self):
        curr = self.sentinel.prev
        while curr != self.sentinel:
            print(curr.data, end=" ⇔ ")
            curr = curr.prev
        print("(sentinel)")

    def is_empty(self):
        return self.sentinel.next == self.sentinel

```

### 3.3 Demo

```

cdll = CDLL()
cdll.insert_back(10)
cdll.insert_back(20)
cdll.insert_back(30)
cdll.insert_front(5)
cdll.traverse_forward()    # 5 ⇔ 10 ⇔ 20 ⇔ 30 ⇔ (sentinel)
cdll.traverse_backward()   # 30 ⇔ 20 ⇔ 10 ⇔ 5 ⇔ (sentinel)

```

---

## 4. Complexity Comparison

| Operation         | CSLL (with tail ptr) | CDLL (with sentinel) |
|-------------------|----------------------|----------------------|
| Insert front      | O(1)                 | O(1)                 |
| Insert back       | O(1)                 | O(1)                 |
| Delete given node | O(n)                 | <b>O(1)</b>          |
| Traverse forward  | O(n)                 | O(n)                 |
| Traverse backward | Not possible         | O(n)                 |

---

## 5. Real-World Applications of Circular Lists

| Application                   | Why Circular?                               |
|-------------------------------|---------------------------------------------|
| <b>Round-robin scheduler</b>  | Each process gets CPU turn in cycles        |
| <b>Audio/video buffer</b>     | Circular buffer (ring buffer) for streaming |
| <b>Board games (Monopoly)</b> | Players take turns in a cycle               |
| <b>Browser tabs</b>           | Ctrl+Tab cycles through tabs                |
| <b>Multiplayer games</b>      | Turn rotation among players                 |
| <b>Music player</b>           | Looping playlist                            |

### Ring Buffer (Circular Queue) — O(1) enqueue and dequeue

```
class RingBuffer:
    def __init__(self, capacity):
        self.buffer = [None] * capacity
        self.capacity = capacity
        self.head = 0
        self.tail = 0
        self.size = 0

    def enqueue(self, item):
        if self.size == self.capacity:
            raise Exception("Buffer full")
        self.buffer[self.tail] = item
        self.tail = (self.tail + 1) % self.capacity
        self.size += 1

    def dequeue(self):
        if self.size == 0:
            raise Exception("Buffer empty")
        item = self.buffer[self.head]
        self.head = (self.head + 1) % self.capacity
        self.size -= 1
        return item
```

---

## Summary

- **Circular Singly Linked List:** last.next → HEAD; useful for round-robin applications.
- With a **tail pointer**, both front and back insertions become O(1).

- **Circular Doubly Linked List:** combines CDLL bidirectionality with circular structure.
  - A **sentinel node** simplifies implementation — no NULL checks needed.
  - CDLL with sentinel:  $O(1)$  insert/delete anywhere given node reference.
- 

## References

- Cormen et al., *Introduction to Algorithms*, Ch. 10.2 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.1 (Springer, 2020)